Generación de código
Sentencias y declaraciones

Actualización de sintactico.y

```
%union {
        char *str;
        ListaC codigo;
}
```

```
%type <codigo> expression statement statement_list print_item
                          print_list read_list
```

Generación de código de asignaciones (1)

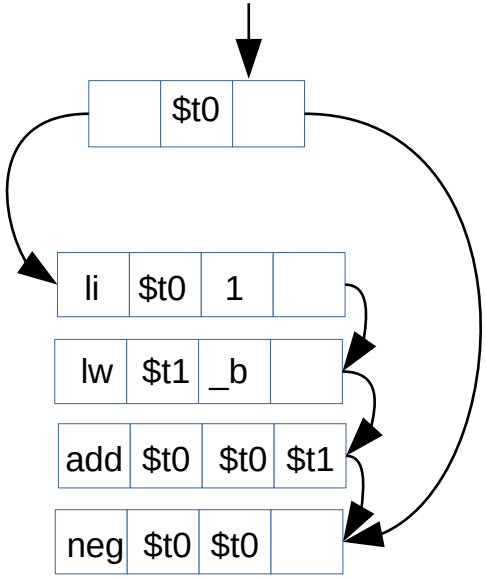
```
void main() {

    var int a;
    const int b = 3;

    a = -(1 + b);

}
```

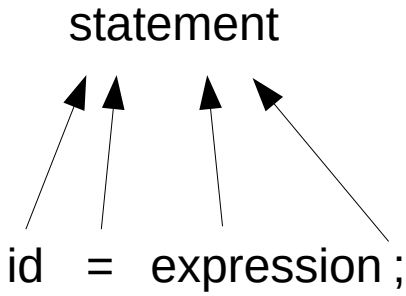
registros

1	0	0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9

statement → id = expression ; { ?? }

$$ $1 → a $3





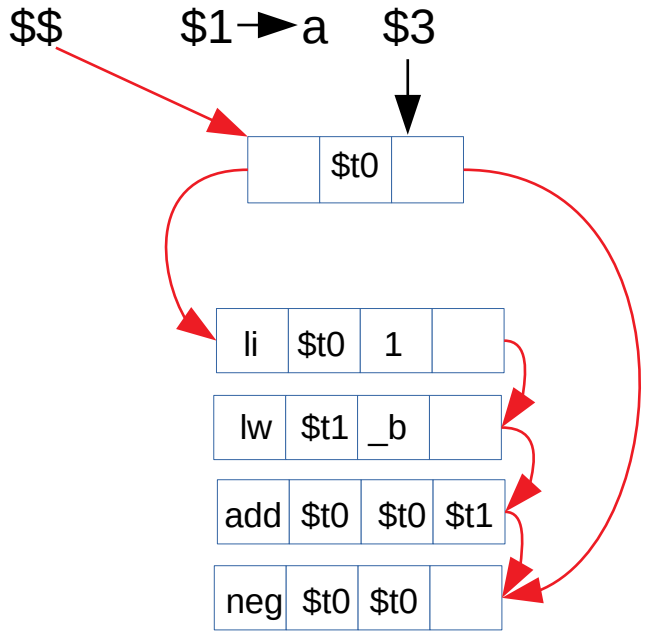
Generación de código de asignaciones (2)

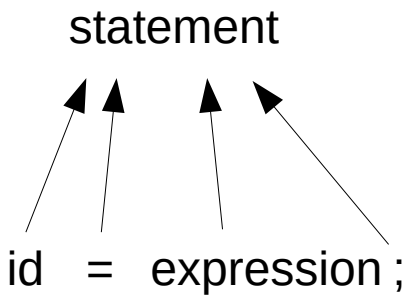
```
void main() {

    var int a;
    const int b = 3;

    a = -(1 + b);

}
```

registros

1	0	0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9

statement → id = expression ; { ?? }



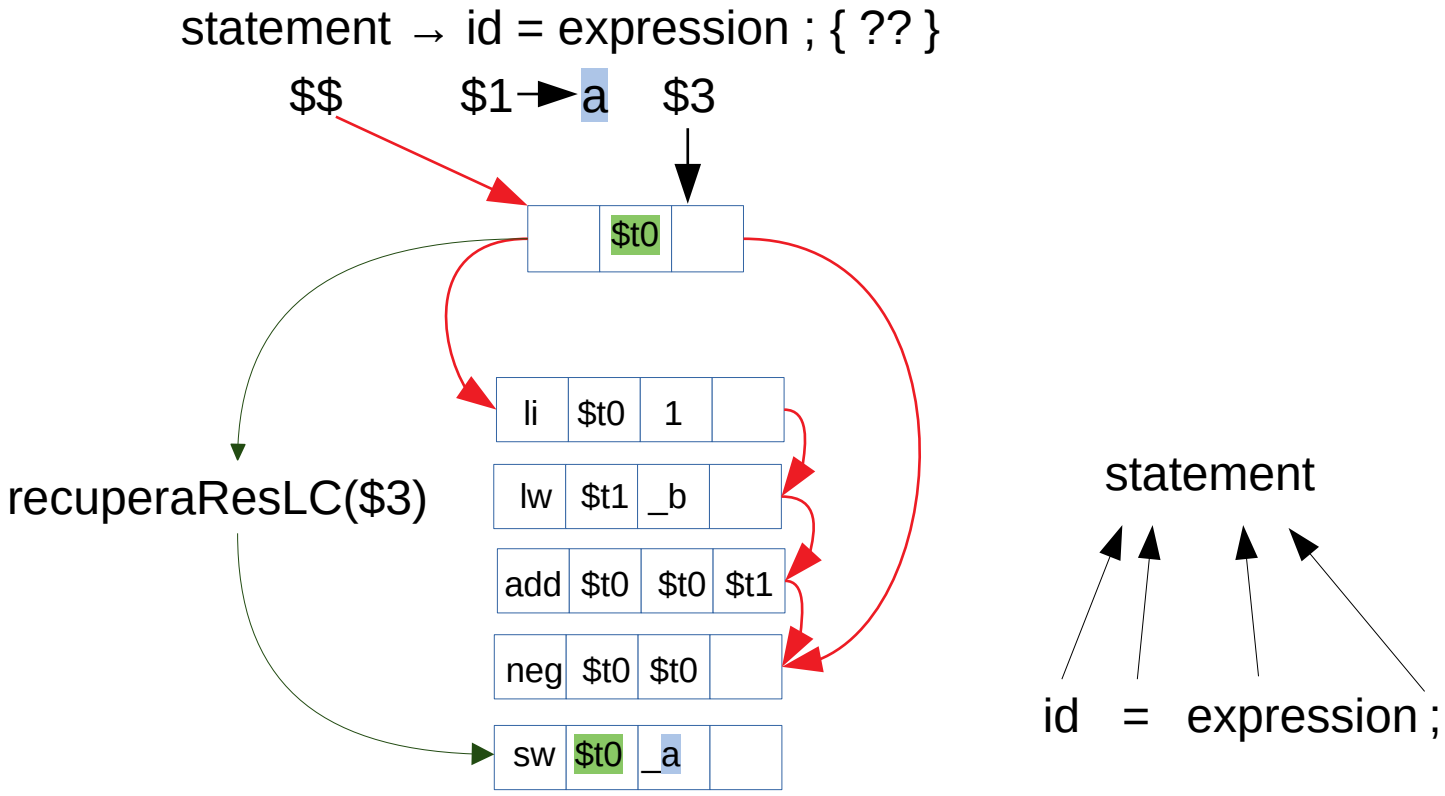


Generación de código de asignaciones (3)

```
void main() {

        var int a;
        const int b = 3;

        a = -(1 + b);

}
```

registros

1	0	0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9



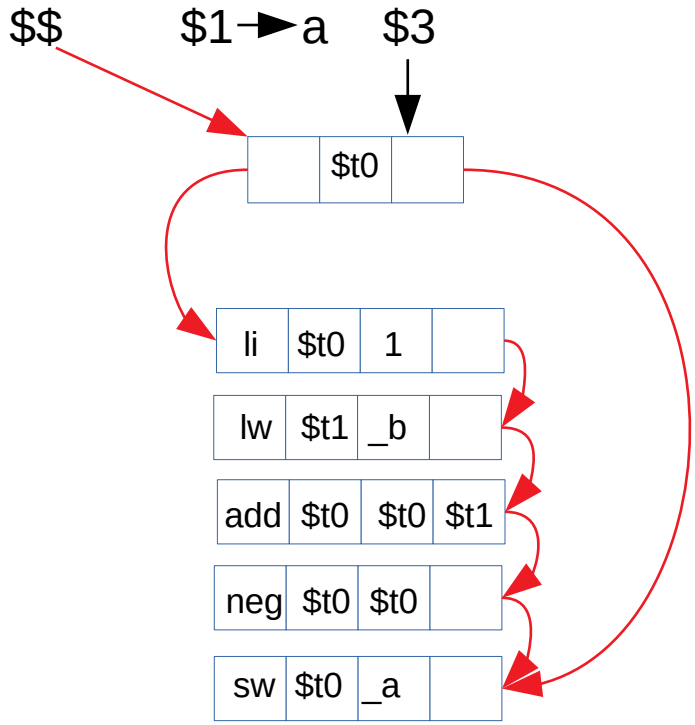
Generación de código de asignaciones (4)

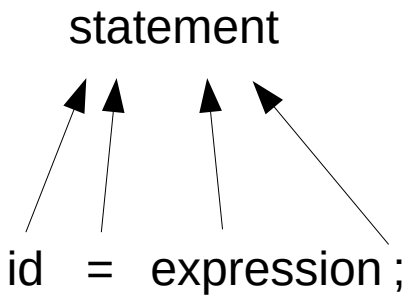
```
void main() {

        var int a;
        const int b = 3;

        a = -(1 + b);

}
```

registros

1	0	0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9

statement → id = expression ; { ?? }





Generación de código de asignaciones (5)

```
void main() {

    var int a;
    const int b = 3;

    a = -(1 + b);

}
```

registros

0	0	0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9

statement → id = expression ; { ?? }

$$ $1 → a $3

¡No olvidéis liberar el registro de la expresión!

	$t0		

li	$t0	1	
lw	$t1	_b	
add	$t0	$t0	$t1
neg	$t0	$t0	
sw	$t0	_a	

statement

id = expression ;

Generación de código de print (1)

- Reglas de producción implicadas:

statement → **print** (print_list) ;
print_list → print_item
| print_list , print_item
print_item → expression
| *string*

Generación de código de print (2)

- Código MIPS para salida por consola de strings:

miniC

```
print ("Respuesta:");
```

MIPS

```
.data
...
$str1: .asciiz "Respuesta:"
...
.text
...
li $v0, 4
la $a0, $str1
syscall
```

Generación de código de print (3)

- Código MIPS para salida por consola de enteros:

miniC

```
print (42);
```

MIPS

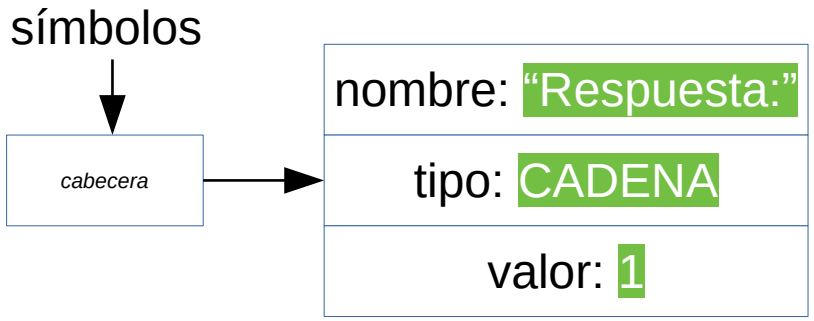
```
li $t0,42
li $v0,1
move $a0, $t0
syscall
```

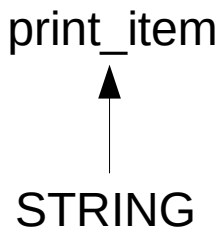
Generación de código de print (4)

```
void main() {

        print ("Respuesta:",42);

}
```

print_item → *string* { ?? }

$$ $1 → "Respuesta:"



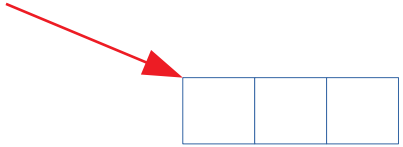


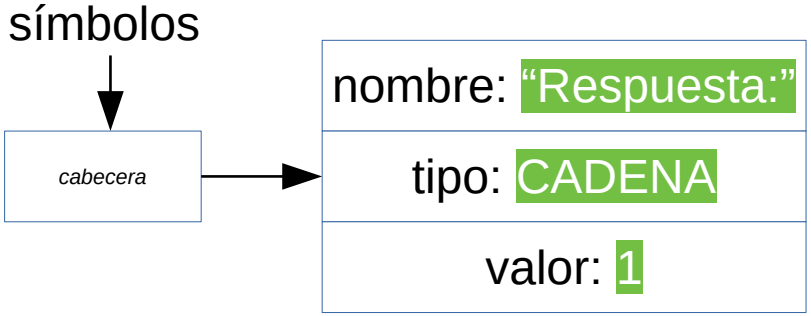
Generación de código de print (5)

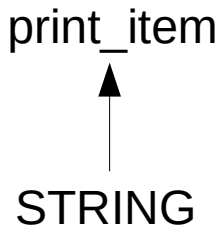
```
void main() {

        print ("Respuesta:",42);

}
```

print_item → *string* { ?? }

$$ $1 → "Respuesta:"





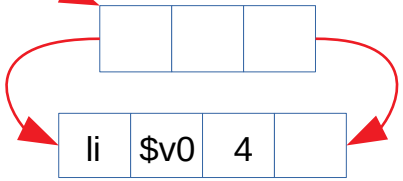


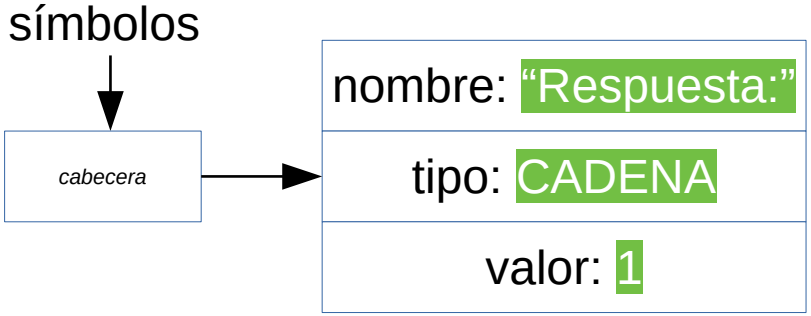
Generación de código de print (6)

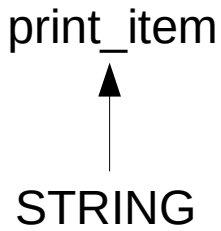
```
void main() {

    print ("Respuesta:",42);

}
```

print_item → *string* { ?? }

$$ $1 → "Respuesta:"





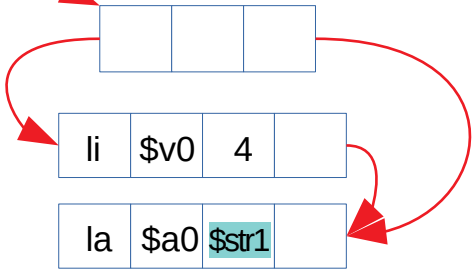


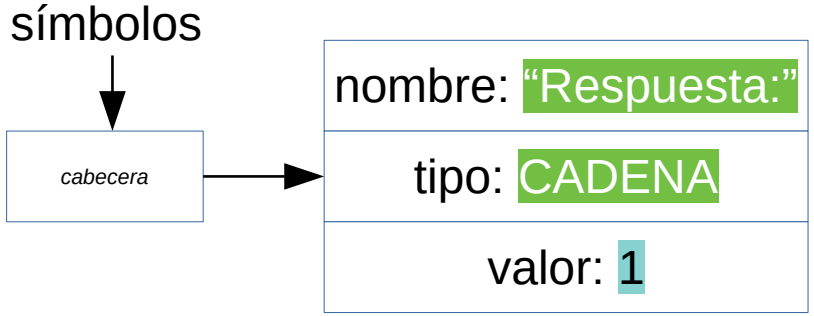
Generación de código de print (7)

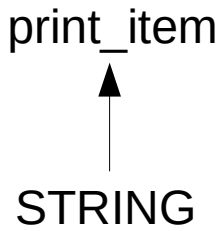
```
void main() {

    print ("Respuesta:",42);

}
```

print_item → *string* { ?? }

$$ $1 → "Respuesta:"





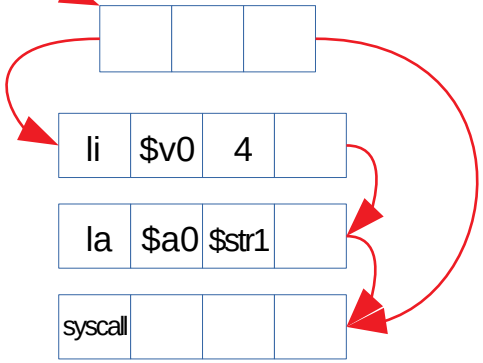


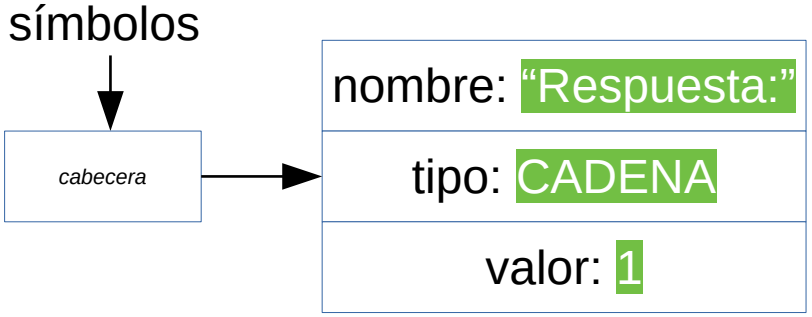
Generación de código de print (8)

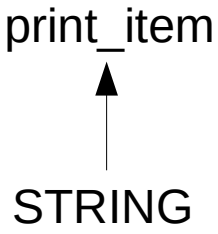
```
void main() {

    print ("Respuesta:",42);

}
```

print_item → *string* { ?? }

$$ $1 → "Respuesta:"





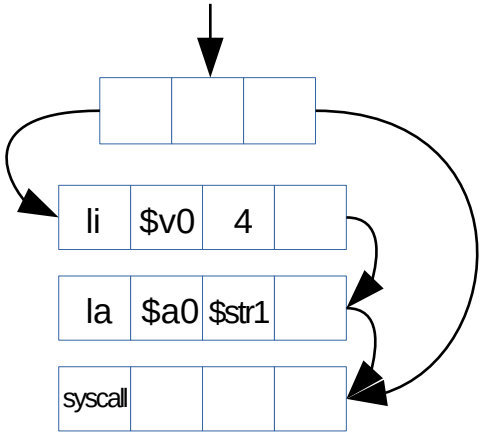


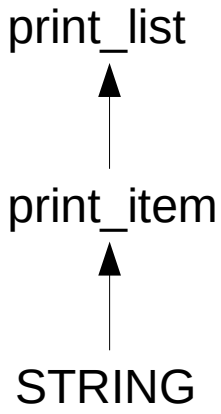
Generación de código de print (9)

```
void main() {

        print ("Respuesta:",42);

}
```

print_list → **print_item** { ?? }

$$ $1

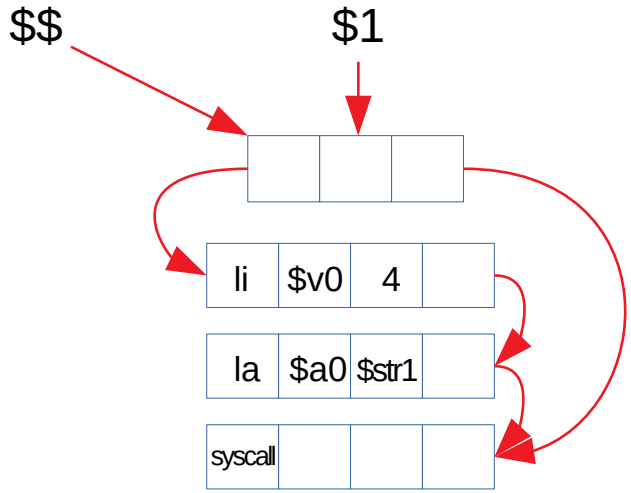


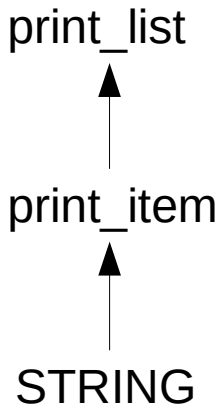


Generación de código de print (10)

```
void main() {

        print ("Respuesta:",42);

}
```

print_list → **print_item** { ?? }



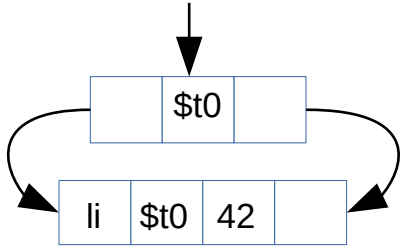


Generación de código de print (11)

```
void main() {

        print (“Respuesta:”,42);

}
```

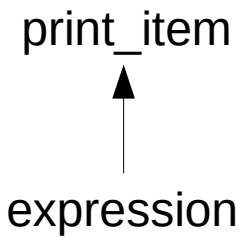
print_item → **expression** { ?? }

$$ $1



registros

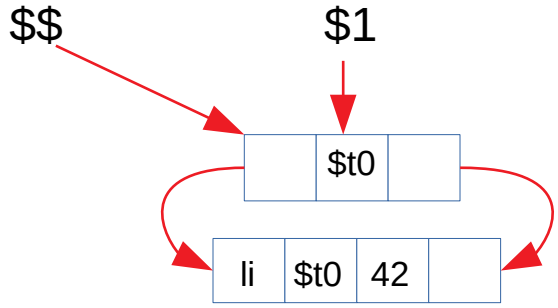
1	0	0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9



Generación de código de print (12)

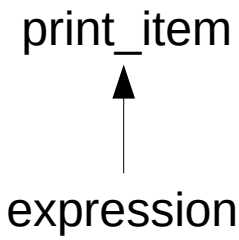
```
void main() {

        print (“Respuesta:”,42);

}
```

print_item → **expression** { ?? }



registros

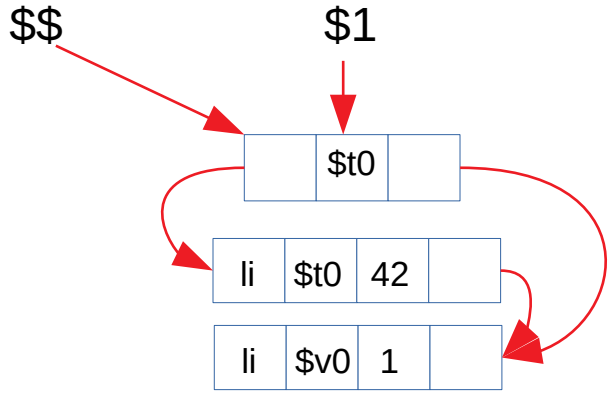
1	0	0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9



Generación de código de print (13)

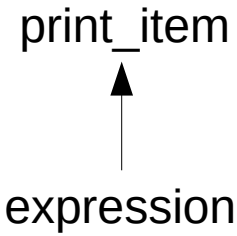
```
void main() {

        print (“Respuesta:”,42);

}
```

print_item → **expression** { ?? }



registros

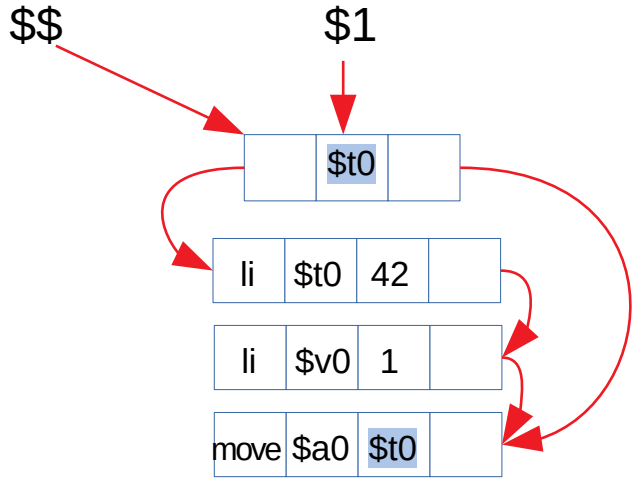
1	0	0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9



Generación de código de print (14)

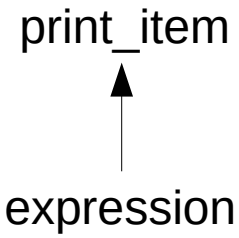
```
void main() {

        print (“Respuesta:”,42);

}
```

print_item → **expression** { ?? }



registros

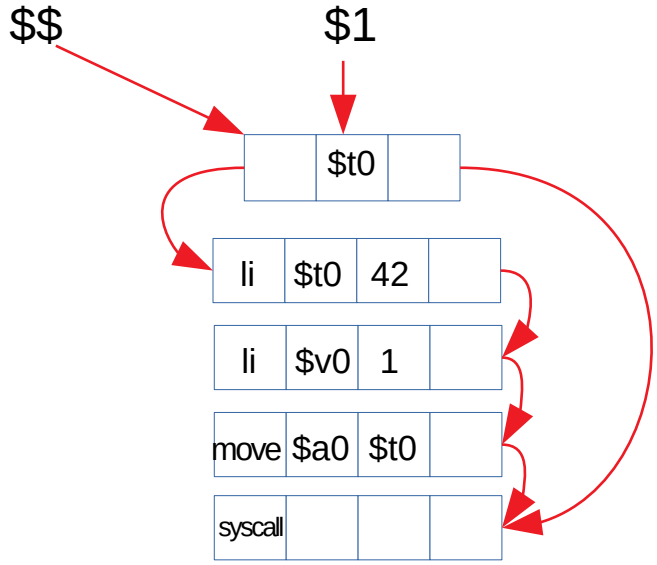
1	0	0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9



Generación de código de print (15)

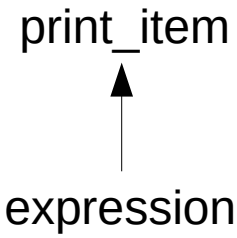
```
void main() {

        print (“Respuesta:”,42);

}
```

print_item → **expression** { ?? }



registros

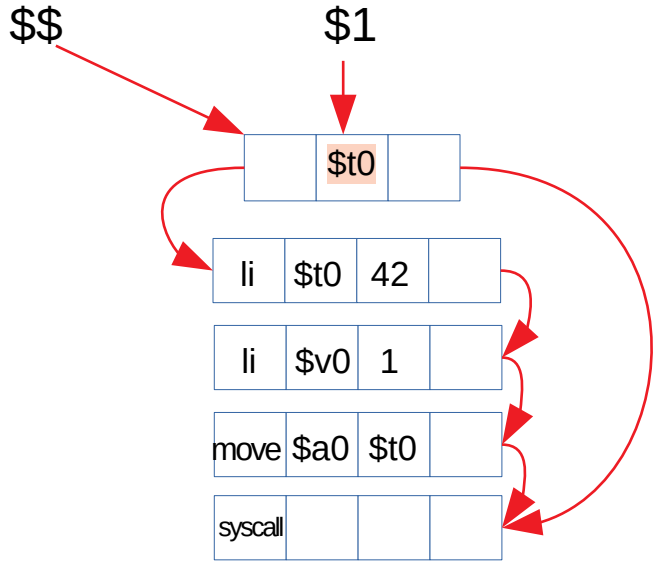
1	0	0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9



Generación de código de print (16)

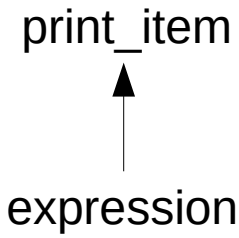
```
void main() {

        print (“Respuesta:”,42);

}
```

print_item → **expression** { ?? }



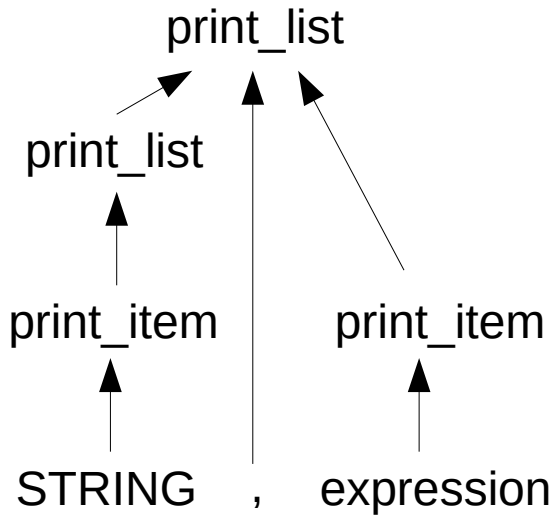
registros

0	0	0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9

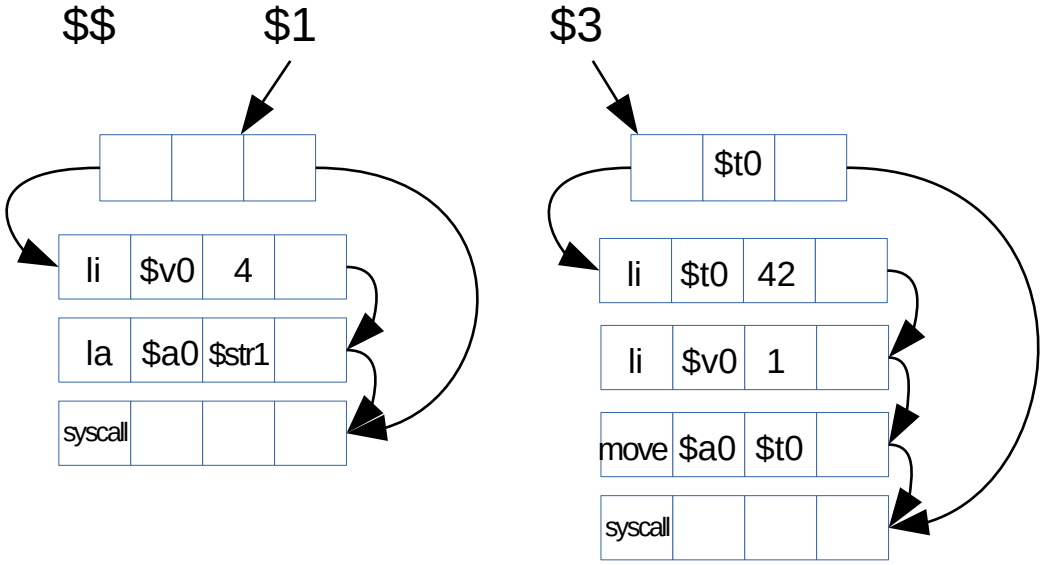


Generación de código de print (17)

```
void main() {
    print ("Respuesta:",42);
}
```

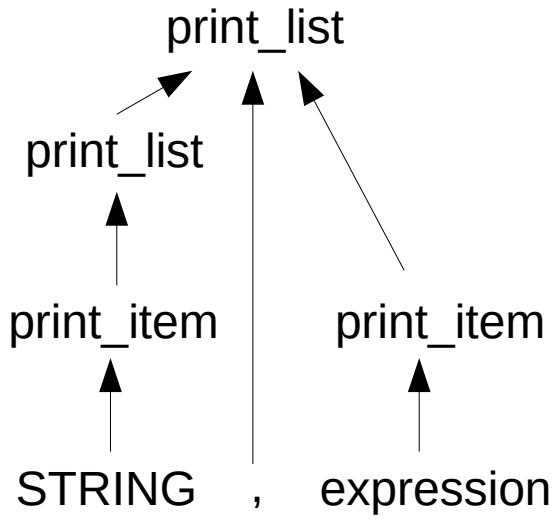


print_list → **print_list , print_item** { ?? }

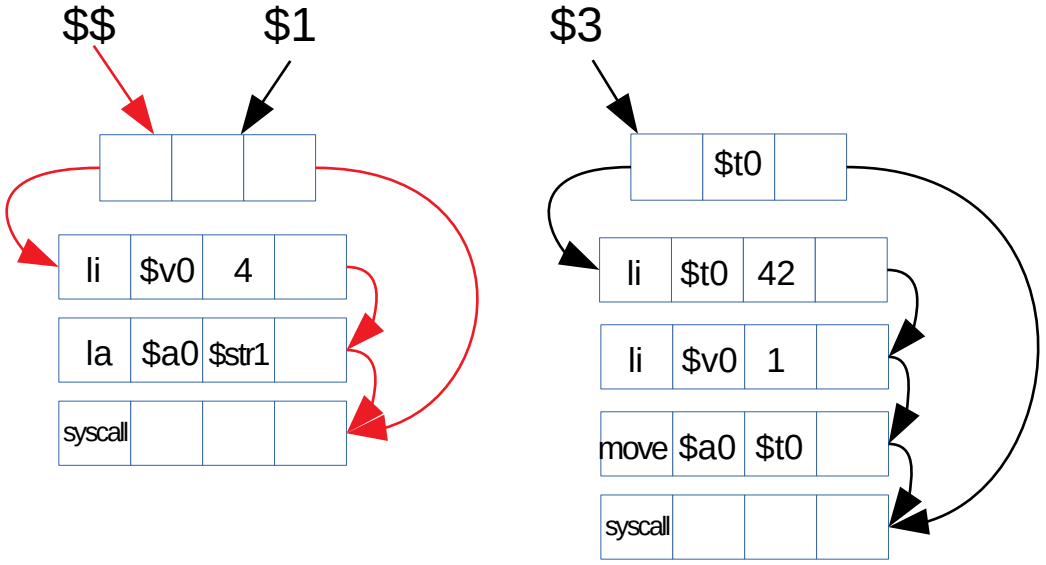


Generación de código de print (18)

```
void main() {

        print ("Respuesta:",42);

}
```

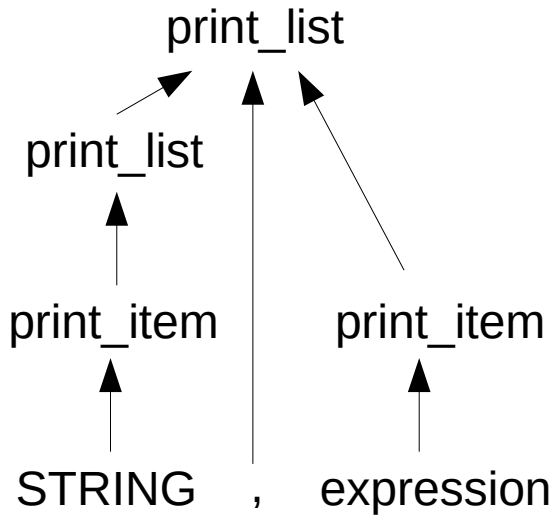


print_list → **print_list , print_item** { ?? }

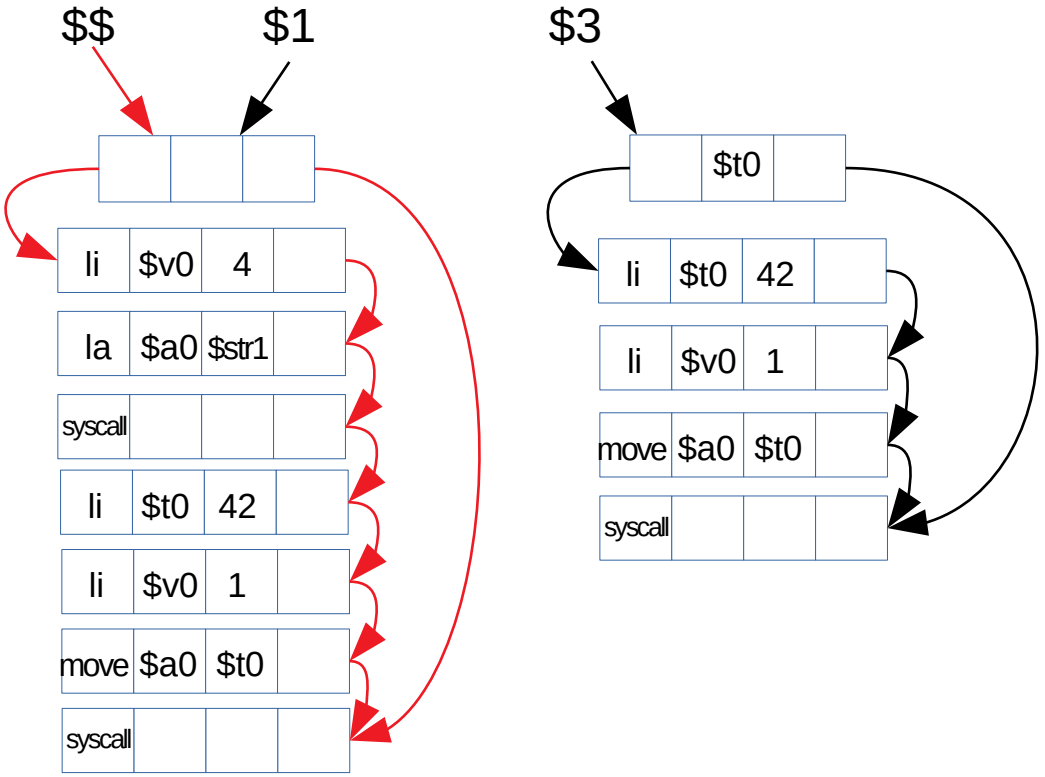


Generación de código de print (19)

```
void main() {
    print ("Respuesta:",42);
}
```

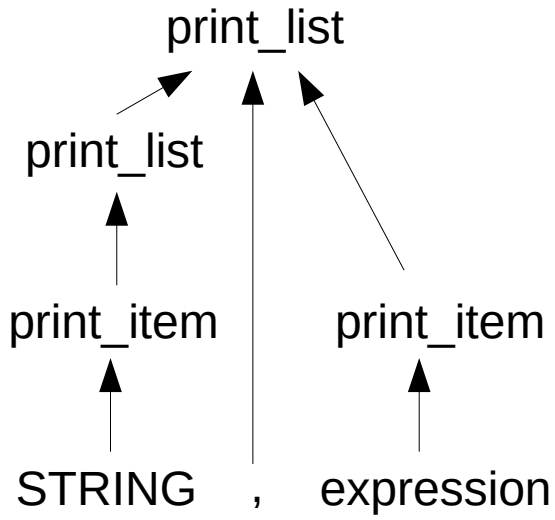


print_list → **print_list , print_item** { ?? }

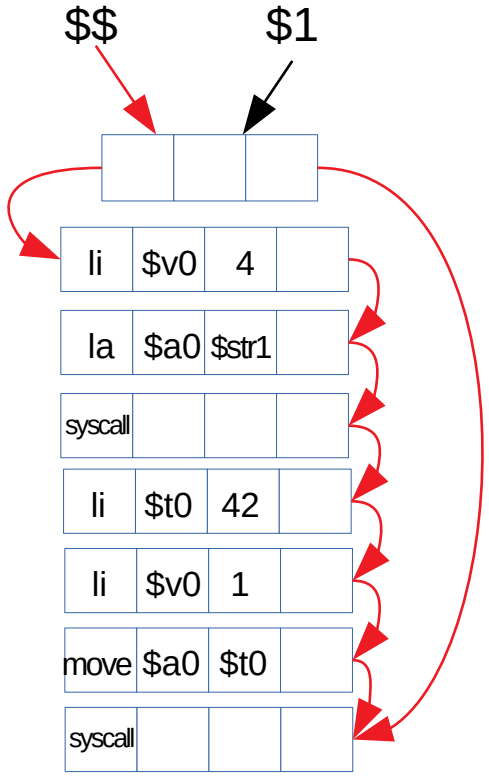


Generación de código de print (20)

```
void main() {
    print ("Respuesta:",42);
}
```

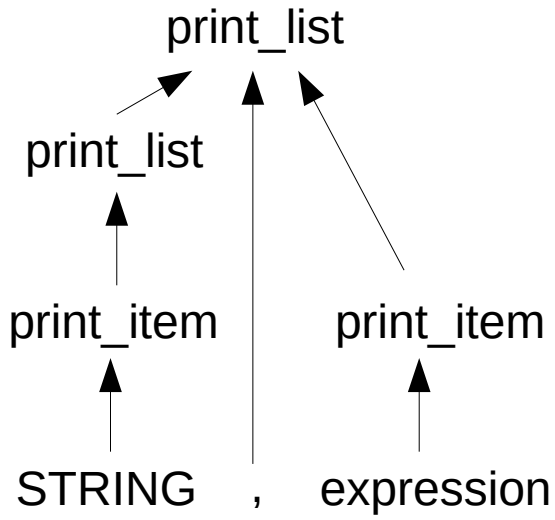


print_list → **print_list , print_item** { ?? }

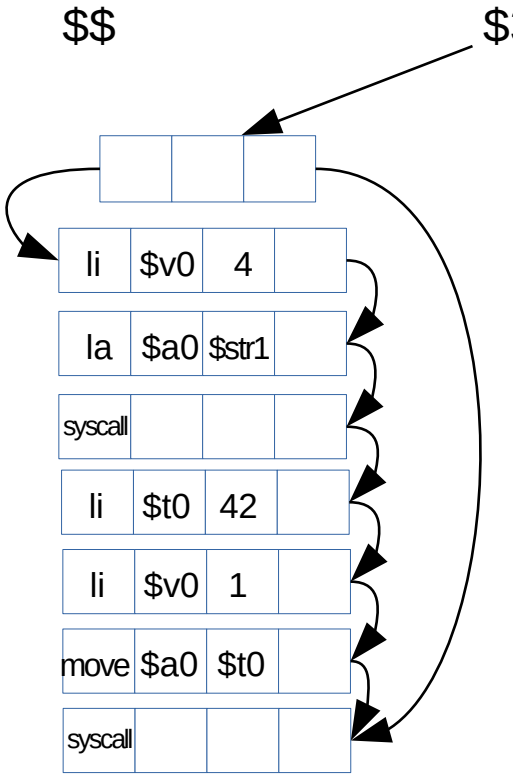


Generación de código de print (21)

```
void main() {
    print ("Respuesta:",42);
}
```

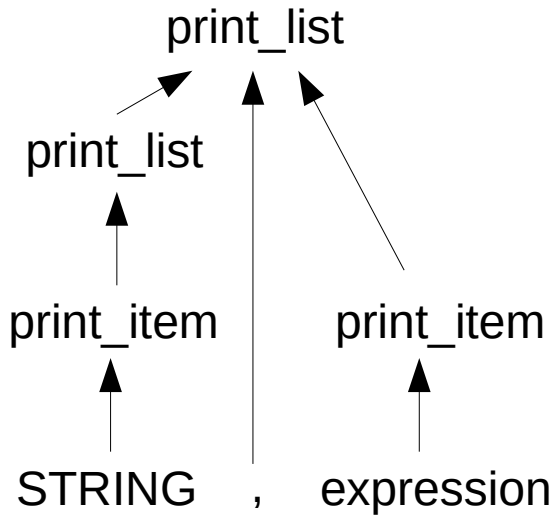


statement → print (**print_list**) ; { ?? }

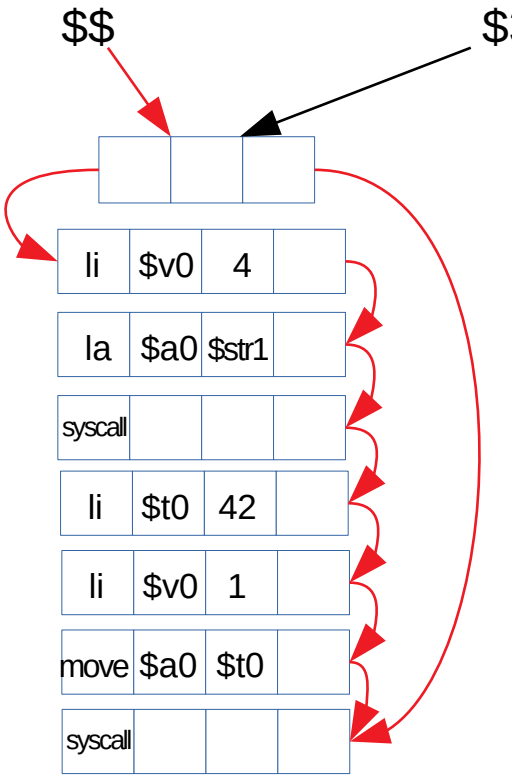


Generación de código de print (22)

```
void main() {
    print ("Respuesta:",42);
}
```



statement → print (**print_list**) ; { ?? }



Generación de código de read (1)

- Reglas de producción implicadas:

statement → read (read_list) ;
read_list → *id*
| read_list , *id*

Generación de código de read (2)

- Código MIPS para salida por consola de enteros:

miniC

```
read (a,b);
```

MIPS

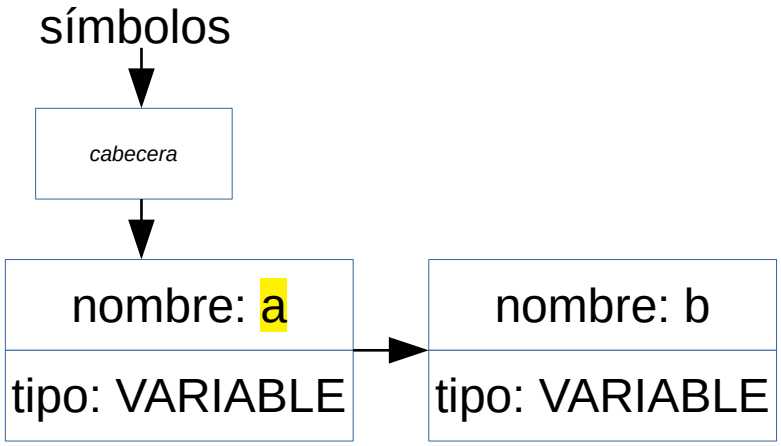
```
li $v0,5
syscall
sw $v0, _a
li $v0,5
syscall
sw $v0, _b
```

Generación de código de read (3)

```
void main() {
        var int a,b;

        read ( a,b );
}
```

read_list → **id** { ?? }

$$ $1 → "a"

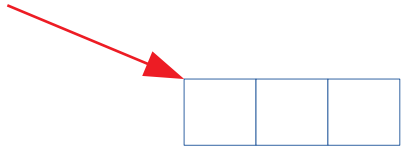


Generación de código de read (4)

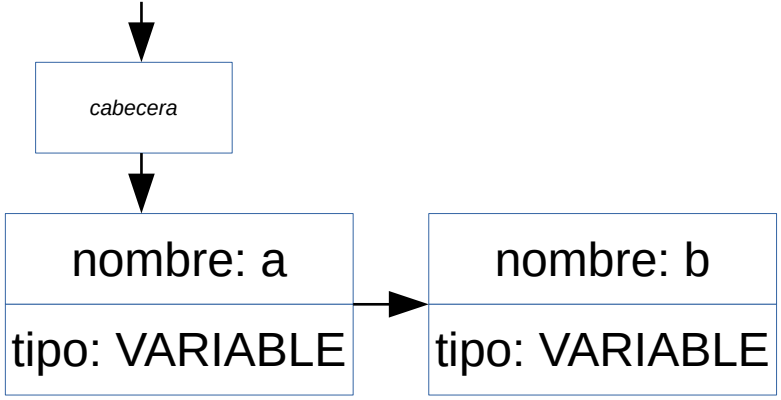
```
void main() {
        var int a,b;

        read ( a,b );
}
```

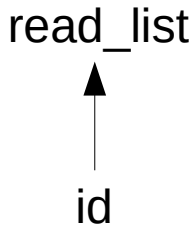
read_list → **id** { ?? }

$$ $1 → "a"



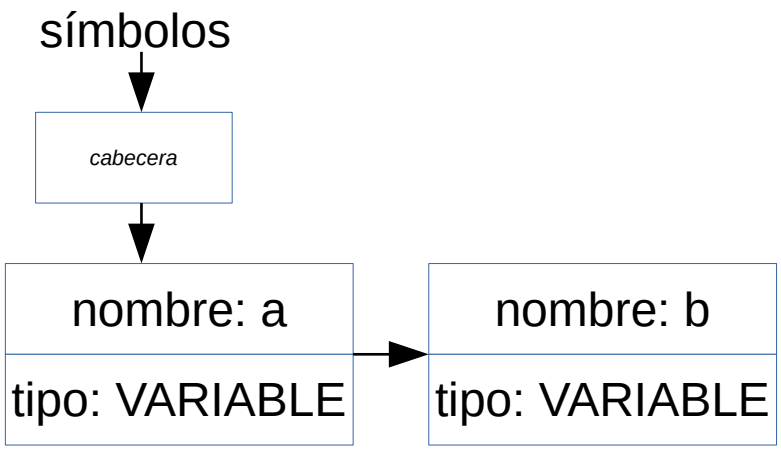
símbolos





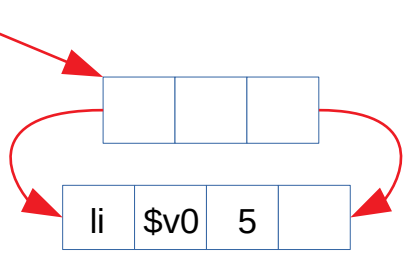
Generación de código de read (5)

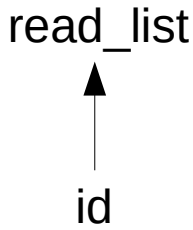
```
void main() {
        var int a,b;

        read ( a,b );
}
```



read_list → **id** { ?? }

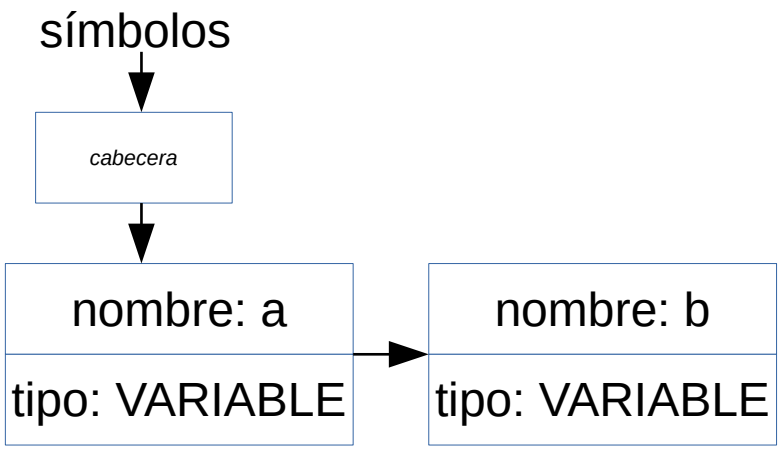
$$ $1 → "a"





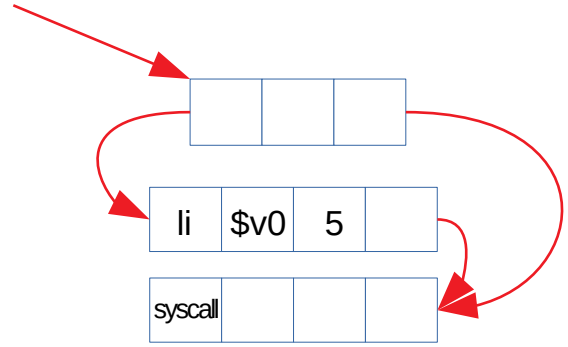
Generación de código de read (6)

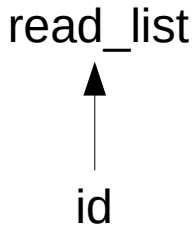
```
void main() {
        var int a,b;

        read ( a,b );
}
```



read_list → **id** { ?? }

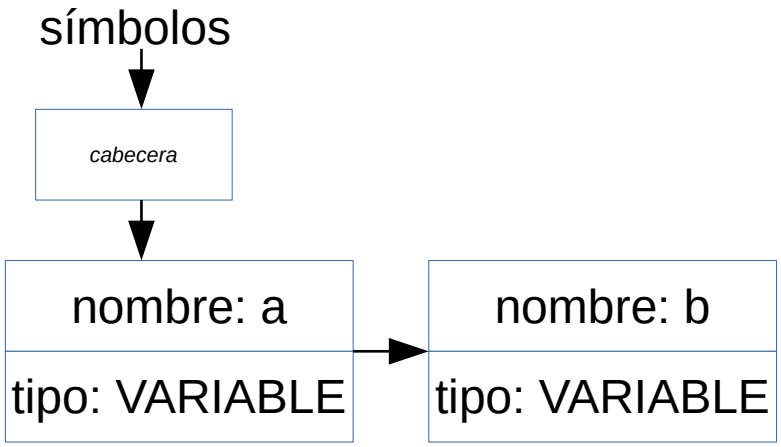
$$ $1 → "a"





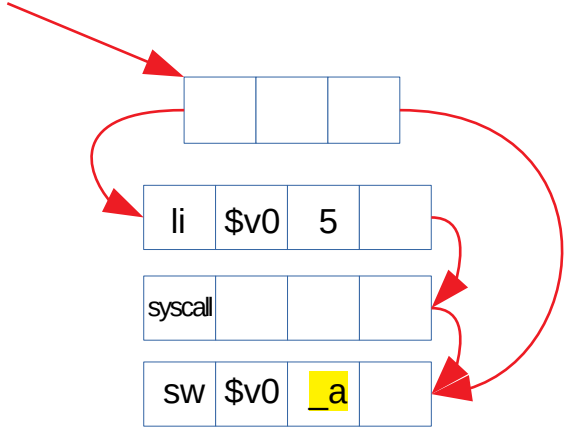
Generación de código de read (7)

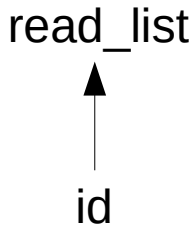
```
void main() {
        var int a,b;

        read ( a,b );
}
```



read_list → **id** { ?? }

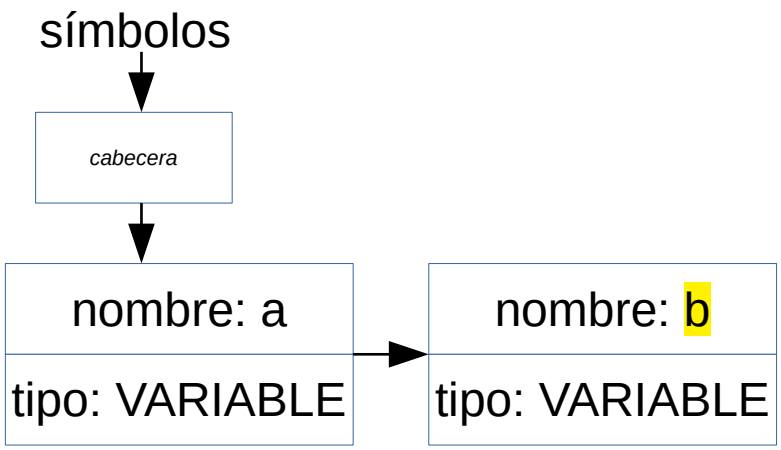
$$ $1 → "a"





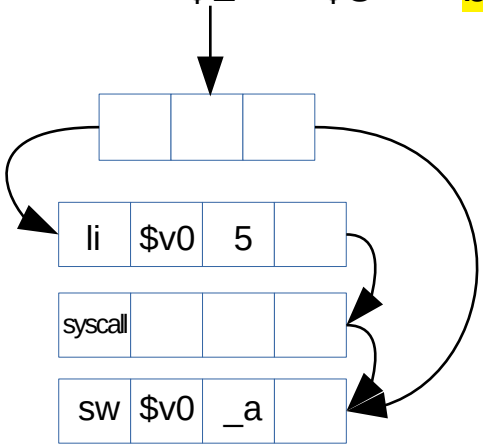
Generación de código de read (8)

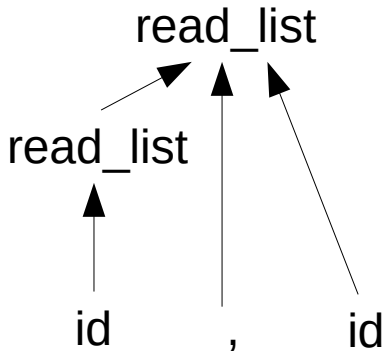
```
void main() {
        var int a,b;

        read ( a,b );
}
```



read_list → **read_list** , **id** { ?? }

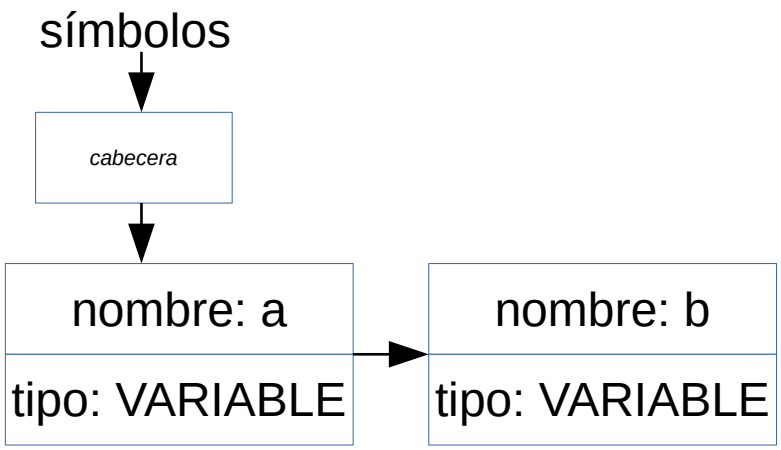
$$ $1 $3 → "b"



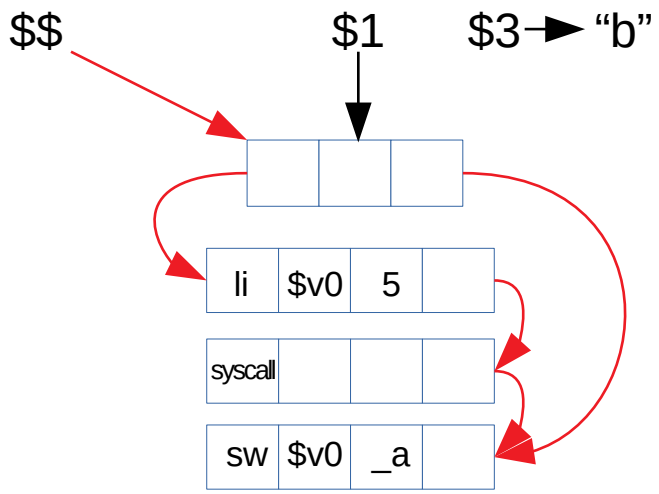


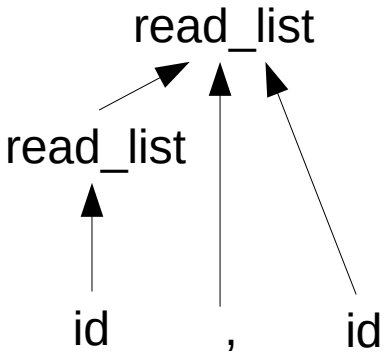
Generación de código de read (9)

```
void main() {
        var int a,b;

        read ( a,b );
}
```



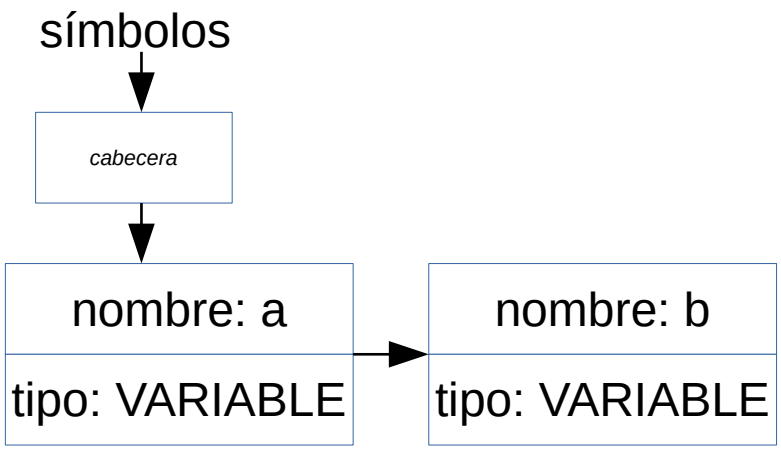
read_list → **read_list** , **id** { ?? }



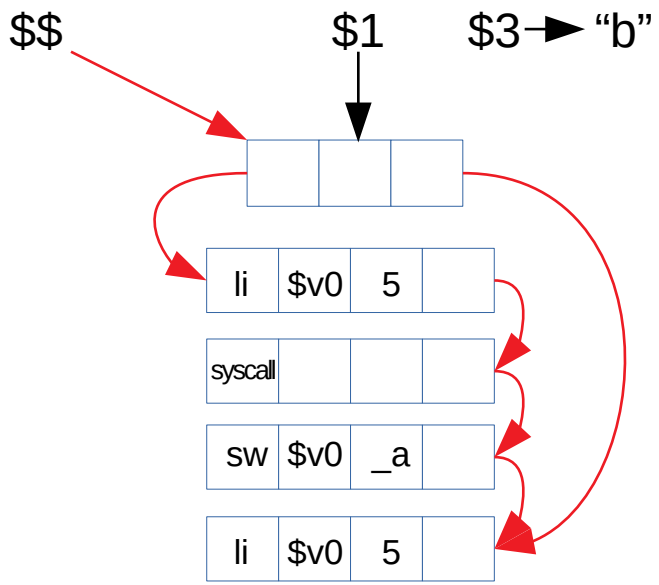


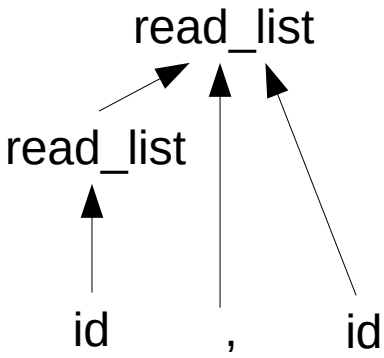
Generación de código de read (10)

```
void main() {
        var int a,b;

        read ( a,b );
}
```



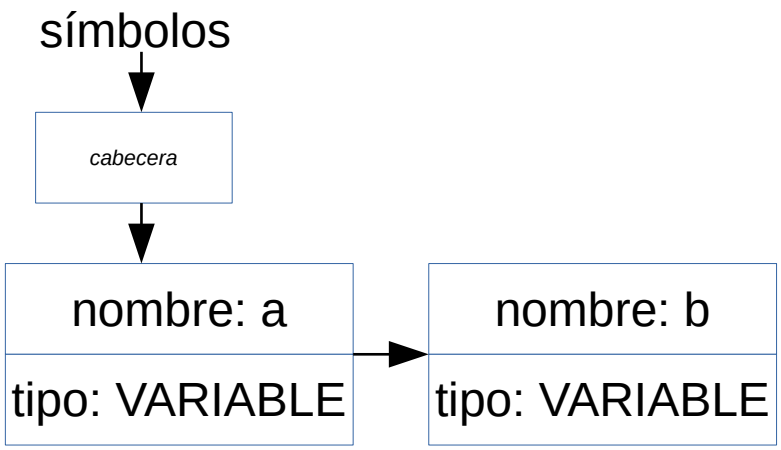
read_list → **read_list** , **id** { ?? }



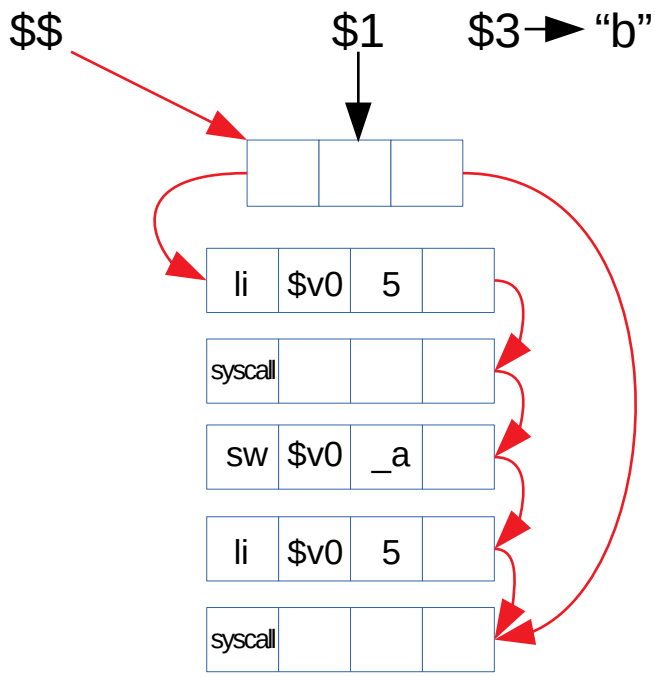


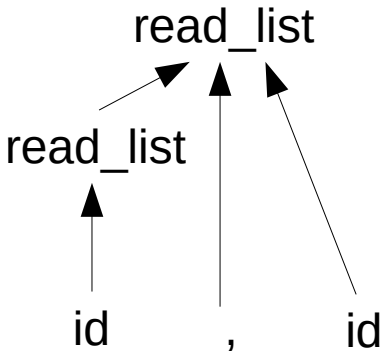
Generación de código de read (11)

```
void main() {
        var int a,b;

        read ( a,b );
}
```



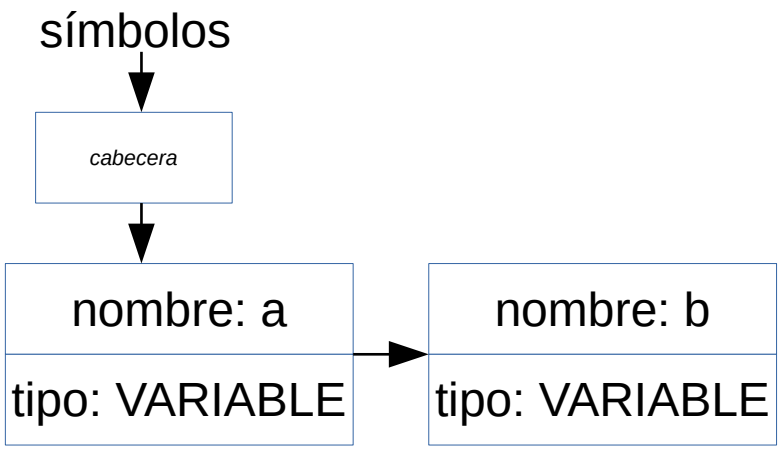
read_list → **read_list** , **id** { ?? }



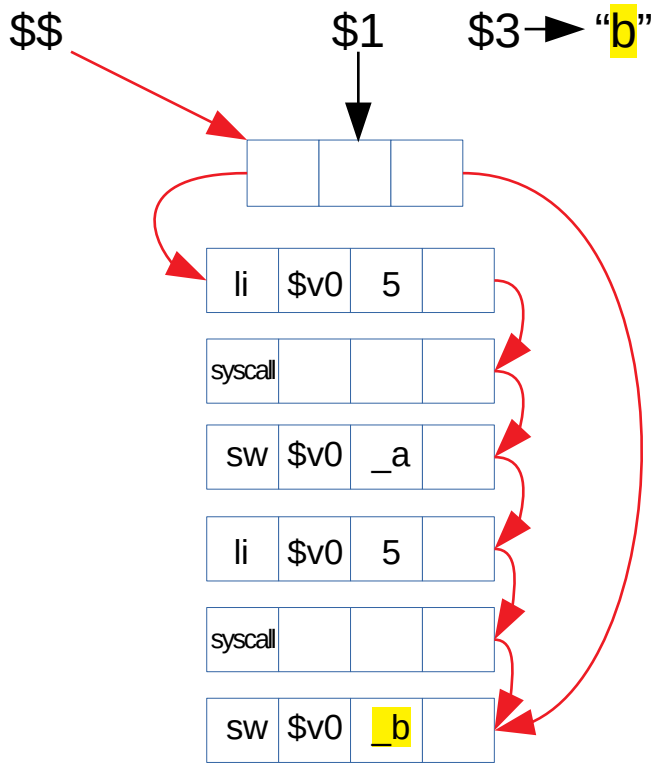


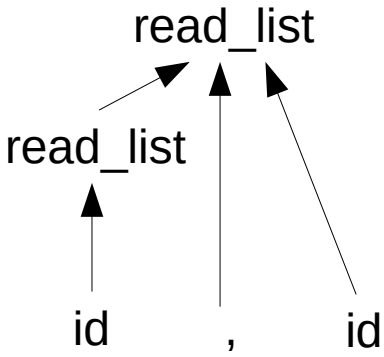
Generación de código de read (12)

```
void main() {
        var int a,b;

        read ( a,b );
}
```



read_list → **read_list** , **id** { ?? }



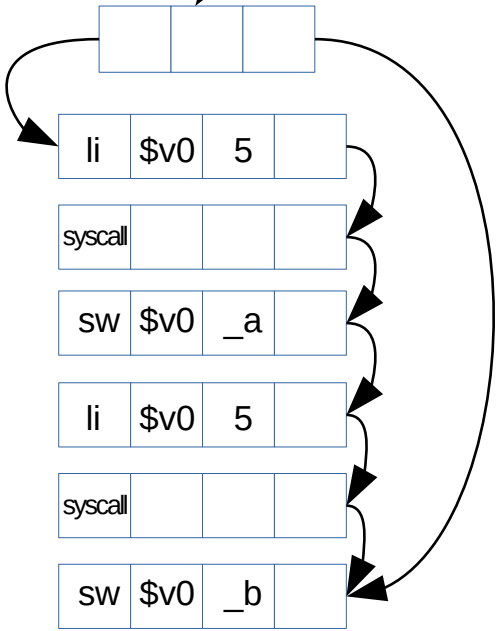


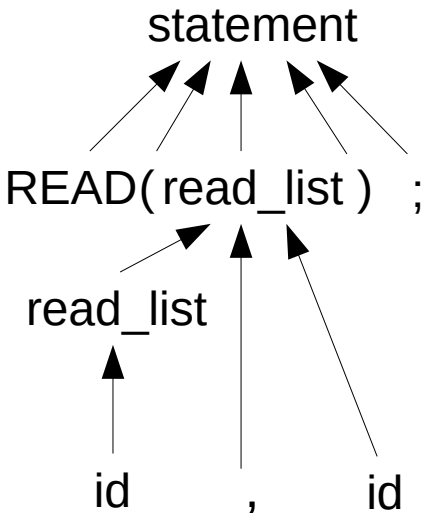
Generación de código de read (13)

```
void main() {
      var int a,b;

      read ( a,b );
}
```

statement → read (**read_list**); { ?? }

$$ $3

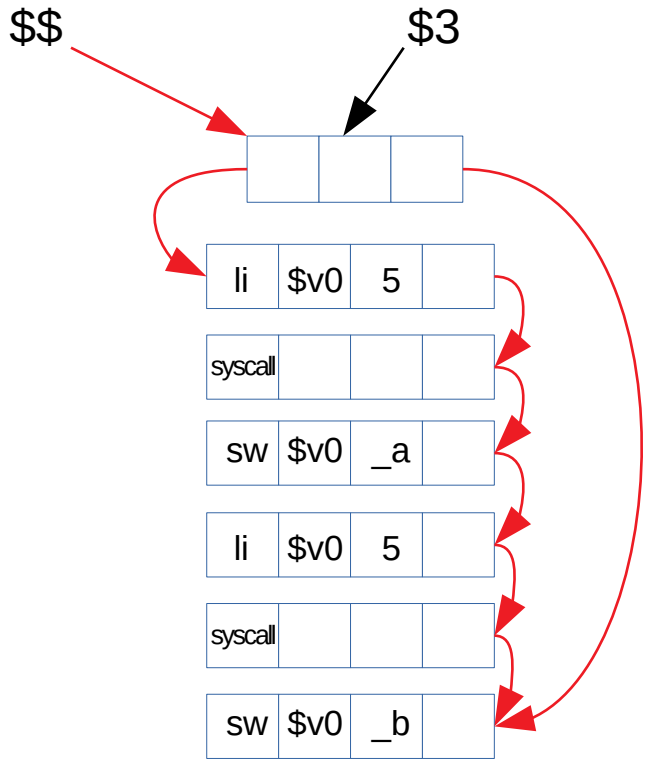


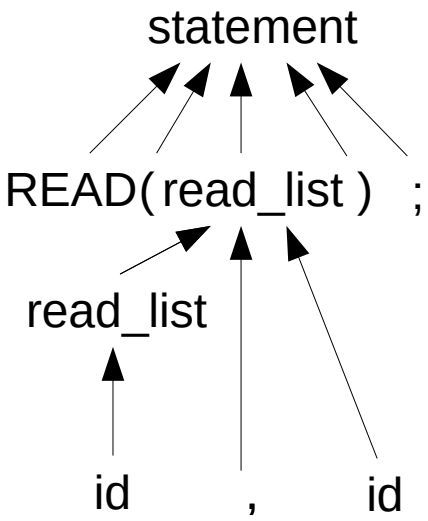


Generación de código de read (14)

```
void main() {
        var int a,b;

        read ( a,b );
}
```

statement → read (**read_list**) ; { ?? }





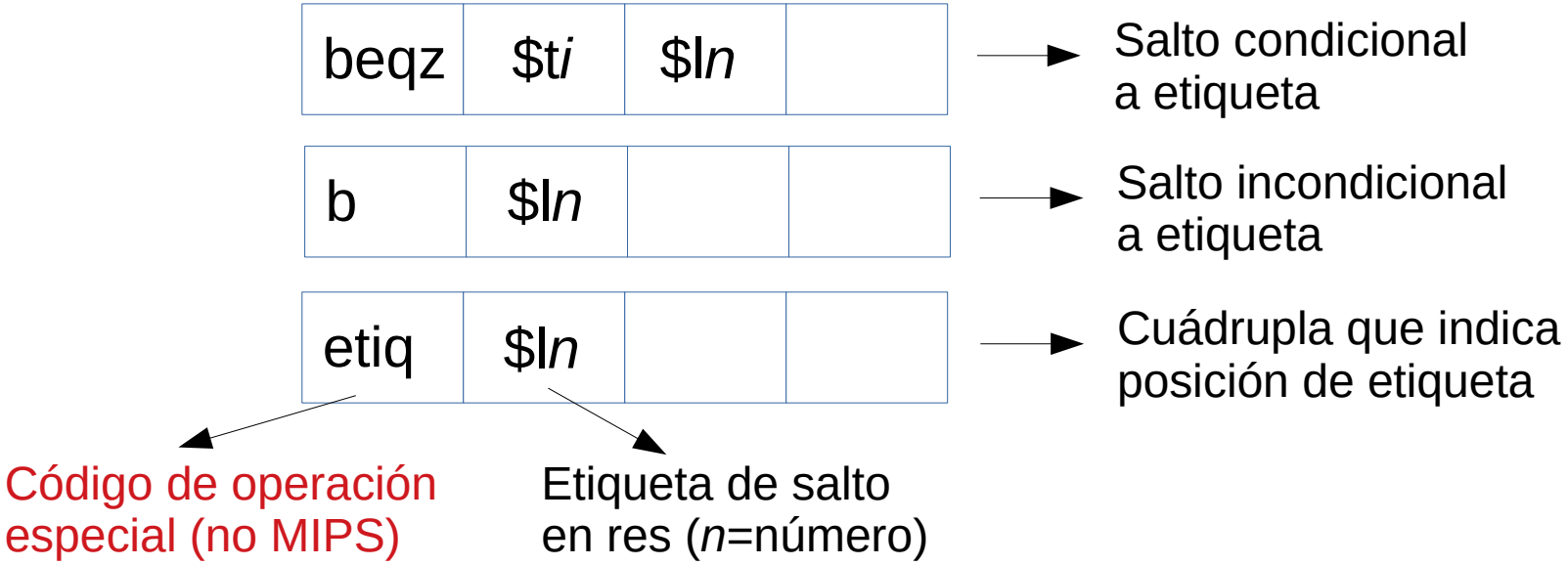
Etiquetas de salto (1)

- La generación de código de sentencias de control de flujo necesita producir etiquetas de salto. Nuevo método en miniC.y:

```
%{ … int contador_etiq = 1; … %}

%%
…
%%

char *nuevaEtiqueta() {
        char *aux;
        asprintf(&aux,”$l%d”,contador_etiq++);
        return aux;
}
```

Etiquetas de salto (2)

- ¿Cómo representamos las etiquetas en la lista de operaciones?



Condiciones

- En miniC, las condiciones son expresiones
- miniC no tienen operadores de comparación (mejora)
- if (expresion) equivale a if (expresion != 0)
- ¡No olvidéis liberar el registro de la expresión!

Generación de código de if (1)

- Regla de producción implicada:

  statement → if (**expression**) **statement**

Generación de código de if (2)

- Código MIPS para if:

miniC

```
if (a)  print ("a") ;
```

expression **statement**

MIPS

```
.data
...
$str1: .asciiz "a"
...
.text
...
        lw $t0, _a          } expression
        beqz $t0, $l1
        li $v0,4            } statement
        la $a0, $str1
        syscall
$l1:
```

Generación de código de if-else (1)

- Regla de producción implicada:

statement → if (**expression**) **statement** else **statement**

Generación de código de if-else (2)

- Código MIPS para if-else:

miniC

```
if (a)  print ("a"); else a=1;
```

expression **statement** **statement**

MIPS

```
.data
...
$str1: .asciiz "a"
...
.text
...
        lw $t0, _a          } expression
        beqz $t0, $l1
        li $v0,4            } statement
        la $a0, $str1
        syscall
        b $l2
$l1:
        li $t1,1            } statement
        sw $t1,_a
$l2:
```

Generación de código de while (1)

- Regla de producción implicada:

  statement → while (**expression**) **statement**

Generación de código de while (2)

- Código MIPS para while:

miniC

```
while (a)   a = a-1;
```

expression **statement**

MIPS

```
.text
...
$l1:
        lw $t0, _a          } expression
        beqz $t0, $l2
        lw $t1,_a           }
        li $t2,1            }
        sub $t1,$t1,$t2     } statement
        sw $t1,_a           }
        b $l1
$l2:
```

Generación de código de statement_list (1)

- Reglas de producción implicadas:

statement → **{** statement_list **}**
statement_list → statement_list statement
| λ

%empty

Generación de código de statement_list (2)

- Ideas para la traducción:

```
while (a)
{
        print ("a=",a,"\n");
        a = a-1;
}
```

statement	→ **{** statement_list **}**	{ /* Pasar a $$ el código de $2 */ }
statement_list	→ statement_list statement	{ /* Pasar a $$ el código de $1 y concatenar en $$ el código de $2 liberar código de $2 */ }
	| λ	{ /* Poner en $$ lista código vacía */ }

Nos acercamos al final...

- En la regla de program se puede imprimir el código del body:

program → **id ( ) {** body **}**

```
{     /* Comprobar si todo ha ido bien, y en ese caso: */
      imprimirLS(ts);    // Volcar .data del programa
      imprimirLC($5);  // Volcar .text del programa
}
```

ATENCIÓN: falta generar el código de body, con la inicialización de identificadores
Si hay una acción en mitad de la regla, se le asigna un $i y desplaza +1 el resto

¿Las declaraciones generan código?

- Sí, en la asignación de valores iniciales...

```
test4() {

    var int a;
    const int b = 3;

    print ("b",b);

}
```

Generación de código de asignación

```
li $t0, 3
sw $t0,_b
```

registros

1	0	0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9

Reglas de producción usadas en la generación de código de declaraciones

declaration → **var** tipo id_list **;**
| **const** tipo id_list **;**

id_list → id_decl
| id_list **,** id_decl

id_decl → id
| id = expression

```
%type <codigo> ... declaration id_list id_decl body
```

Ideas para id_decl

```
id_decl → id {
            /*   1. Verificación semántica de $1
                 2. $$ = lista vacía
          }

        | id = expression {
            /*   1. Verificación semántica de $1
                 2. $$ = código de asignación
                 3. Liberar registro de $3 */
            }
```

Ideas para id_list

```
id_list → id_decl {
            /*  1. Asignar código de $1 a $$ */
          }

          |  id_list , id_decl  {
            /*  1. Asignar código de $1 a $$
                2. Concatenar en $$ el código de $3
                3. Liberar $3
            */
          }
```

Ideas para declaration

```
declaration → var tipo id_list ;
              {
                  /*  1. Asignar código de $3 a $$ */
              }
            | const tipo id_list ;
              {
                  /*  1. Asignar código de $3 a $$ */
              }
```

Finis coronat opus !

- En la regla de body:

```
body  →  body declaration { /* Concatenar $1 y $2 */ }
       |  body statement  { /* Concatenar $1 y $2 */ }
       |  λ                { /* Lista vacía */ }
```

ATENCIÓN: ¡Hay que comprobar que el código ensamblador funciona en MARS o spim!

Final del programa MIPS

El método imprimirLC() debe concluir imprimiendo dos operaciones que realizan el exit del programa:

```
li $v0, 10
syscall
```